

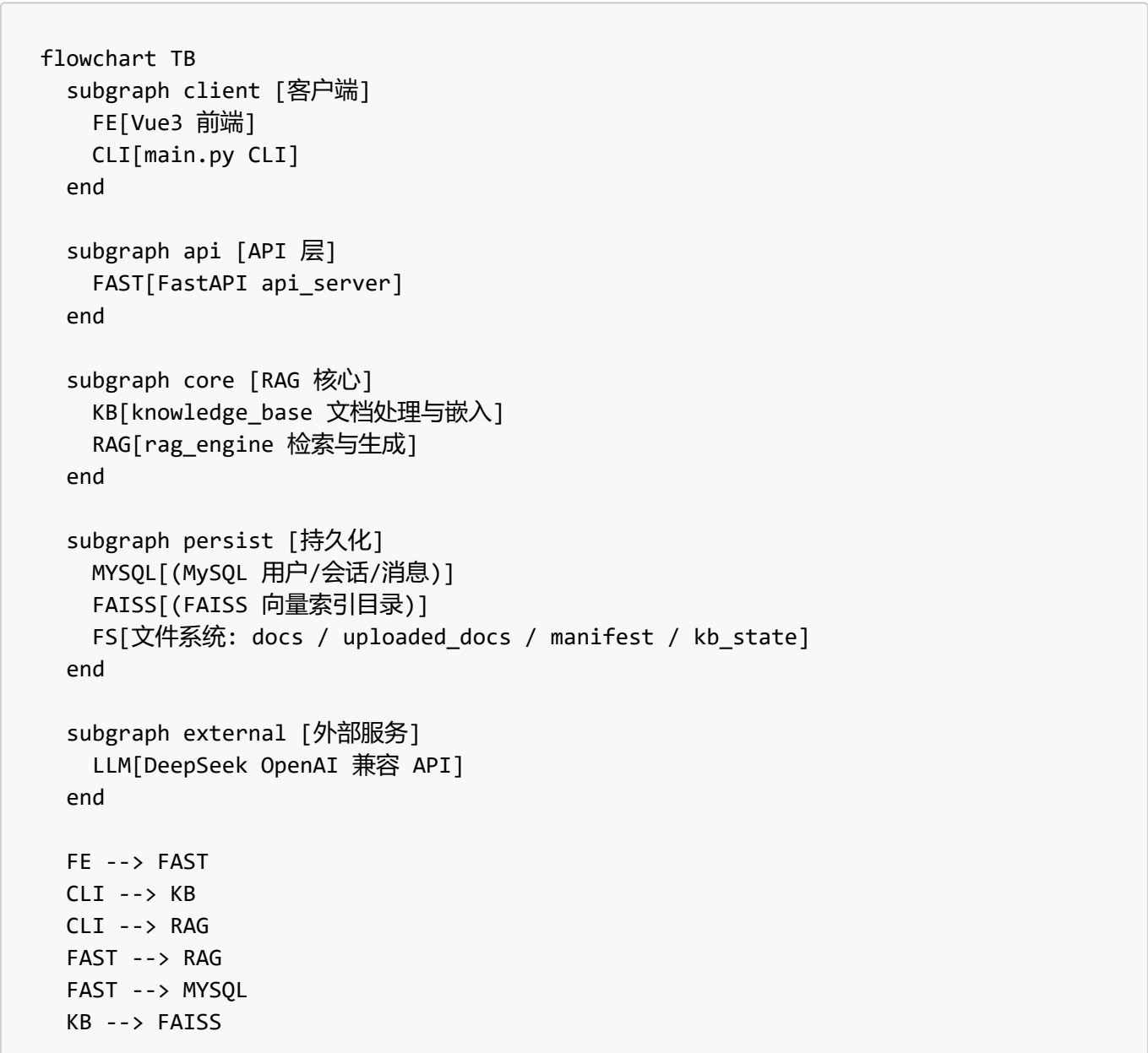
HR RAG 系统 — 业务逻辑与架构说明

本文档梳理 `my_rag_system` 的**业务目标**、**模块边界**、**数据如何流动**以及**存储分层**，便于维护与二次开发。

1. 项目定位

- **业务**：面向企业 HR 制度（或可替换 Prompt 的其他制度类）的**检索增强问答（RAG）**。
- **能力**：用户上传/选定知识文档 → 向量化写入 **FAISS** → 用户提问 → **混合检索**（向量 + BM25，可选 CrossEncoder 重排）→ **DeepSeek** 等大模型生成回答（含「推理 + 最终答案」结构化输出）。
- **形态**：
 - **CLI**： `main.py` 交互式问答。
 - **HTTP API**： `api_server.py`（FastAPI），供 **Vue 3 前端**或第三方调用。
 - **评测**： `ragas/` 下脚本与 RAGAS 指标（如 faithfulness、answer_relevancy）。

2. 总体架构（逻辑分层）



```
RAG --> FAISS
RAG --> LLM
KB --> FS
FAST --> FS
```

要点:

- **向量检索**始终在本地 **FAISS** (目录由 `FAISS_INDEX_PATH` 配置) , **不写入 MySQL**。
- **MySQL** (可选) 仅存 **用户、会话、问答消息** 等业务数据; 未配置 `DATABASE_URL` 时 API 行为与旧版一致, 不落库。
- **静态制度文档**默认在 `docs/` (`DOCS_DIR`) ; **用户上传**在 `uploaded_docs/` (`UPLOADED_DOCS_DIR`) 。

3. 核心模块职责

模块	文件	职责
配置	<code>config.py</code>	环境变量、路径解析 (<code>DOCS_DIR</code> 、 <code>FAISS_INDEX_PATH</code> 、种子文档、 <code>DATABASE_URL</code> 、API/CORS、检索与重排参数、Prompt 模板入口等) 。
知识库	<code>knowledge_base.py</code>	HuggingFace 嵌入; PDF/Word/TXT 加载与清洗; 父子切块 (父块约 1200 字、子块约 300 字, 子块 metadata 带 <code>parent_content</code>) ; <code>create_vector_store</code> / <code>load_vector_store</code> 。
检索与生成	<code>rag_engine.py</code>	向量检索 + BM25 Ensemble ; 可选 CrossEncoder 重排; <code>RetrievalQA</code> 或手写 prompt + LLM; concise 后处理; 推理/最终答案 拆分字段 <code>answer_only</code> ; 支持 流式 <code>stream_callback</code> ; <code>replace_vectorstore</code> 热切换索引。
上传与索引编排	<code>doc_store.py</code>	<code>manifest.json</code> 清单、 <code>kb_state.json</code> 当前选中文档 id; <code>rebuild_faiss_from_doc_ids</code> 多文档合并切块并写 FAISS。
HTTP	<code>api_server.py</code>	生命周期内: <code>init_database</code> → <code>KnowledgeBase</code> → 按 <code>kb_state</code> / 磁盘 FAISS / 种子文档启动 <code>RAGEngine</code> ; 问答、SSE 流式、文档上传/删除/重建索引、会话列表与历史; 健康检查含 DB ping。
数据库	<code>db/</code>	SQLAlchemy 模型: <code>rag_users</code> 、 <code>rag_conversations</code> 、 <code>rag_messages</code> ; 鉴权 X-API-Key (可选) ; 聊天落库。
入口	<code>main.py</code>	无 FAISS 时按 <code>get_seed_document_path()</code> 建库; 交互式流式问答。
评测	<code>ragas/*</code>	构造评测集、调用 <code>RAGEngine.ask</code> 、RAGAS 打分; 与主站配置共用 <code>config</code> 。

4. 业务逻辑流程

4.1 服务启动时 (API)

- 1. **MySQL**: 若配置了 `DATABASE_URL`, 执行 `create_all` 并确保默认用户 `DEFAULT_APP_USER` (可选绑定 `API_SERVICE_API_KEY` 的 SHA256) 。
- 2. **嵌入模型**: KnowledgeBase 加载本地或 Hub 嵌入 (受 `LOCAL_MODEL_PATH` / `EMBEDDING_MODEL_NAME` 等影响) 。
- 3. **向量索引加载顺序** (`_bootstrap_rag_core`) :
 - 若 `kb_state.json` 中 `active_document_ids` 非空且文件齐全 → **按上传文档重建** FAISS;
 - 否则若 `FAISS_INDEX_PATH` 目录已存在 → **加载已有索引**;
 - 否则 → 使用 `get_seed_document_path()` 指向的**种子文档**建库。
- 4. 构造 **RAGEngine(vectorstore)** (内部再建 BM25、Ensemble、可选压缩器、QA 链等) 。

4.2 用户上传文档并限定知识范围

- 1. **上传**: `POST /api/documents/upload` → 文件写入 `uploaded_docs/files/`, `manifest.json` 追加记录。
- 2. **构建知识库**: 前端 (或脚本) 选择若干 `document_id`, 调用 `POST /api/documents/rebuild` → `rebuild_faiss_from_doc_ids` 合并切块、打 `document_id / source_filename` metadata → **覆盖保存 FAISS** → `write_kb_state` → **RAGEngine.replace_vectorstore**。
- 3. 此后问答**仅检索当前 FAISS 中的内容** (即所选文档的并集) 。

4.3 单次问答 (非流式)

- 1. 若启用 DB: 根据 **X-API-Key** (可选) 解析 `user_id`; **写入用户消息**; 无 `conversation_id` 则**新建会话**。
- 2. `RAGEngine.ask(question)`: 查询扩展 (可选) → 检索 → (可选重排/压缩) → LLM → concise 与推理拆分 → 返回 `result / answer_only / source_documents`。
- 3. 若启用 DB: **写入助手消息** (含简要 extra, 如 source 数量) 。

4.4 流式问答 (SSE)

- 1. 同上先落库用户侧并确定 `conversation_id`, SSE 首包可下发 `meta.conversation_id`。
- 2. 后台线程执行 `ask(..., stream_callback)`, 主协程通过队列转发 `delta`。
- 3. 结束后发 **final** (含后处理全文与 `conversation_id`) , 并在工作线程内**写入助手消息**。

4.5 CLI (main.py)

- 逻辑简化: 无索引则种子文档建库; 循环 **ask + 流式打印**; **不落 MySQL** (除非未来接同一套持久化层) 。

5. 存储与数据分层

数据类型	存放位置	说明
向量与 docstore	<code>FAISS_INDEX_PATH</code> 目录	LangChain FAISS <code>save_local</code> ; 检索唯一真源。
静态源文档	<code>DOCS_DIR</code> (默认 <code>docs/</code>)	种子文档、人工放置的制度文件。

数据类型	存放位置	说明
上传文件 与清单	<code>uploaded_docs/files/</code> + <code>manifest.json</code>	上传记录与磁盘路径； 清单未迁 MySQL （可后续演进）。
当前索引 对应的文 档 id	<code>kb_state.json</code>	<code>active_document_ids</code> 列表。
用户 / 会 话 / 消息	MySQL 表 <code>rag_*</code>	仅当 <code>DATABASE_URL</code> 配置时启用。
密钥与模 型路径	<code>.env</code> （勿提交仓库）	<code>DEEPSEEK_API_KEY</code> 、 <code>DATABASE_URL</code> 、 <code>API_SERVICE_API_KEY</code> 等。

6. 配置入口（`.env` + `config.py`）

- **路径类**： `DOCS_DIR`、`FAISS_INDEX_PATH`、`UPLOADED_DOCS_DIR`、`SEED_DOCUMENT_PATH` / `DEFAULT_SEED_DOCUMENT` 等。
- **模型与检索**： `EMBEDDING_*`、`SEARCH_K`、`RERANK_*`、`USE_RERANKER`、`RAG_PROMPT_STYLE` 等。
- **API**： `API_HOST`、`API_PORT`、`API_CORS_ORIGINS`。
- **生产持久化**： `DATABASE_URL`、`DEFAULT_APP_USER`、`API_SERVICE_API_KEY`、`API_REQUIRE_API_KEY`。
- **Prompt**： `RAG_SYSTEM_TYPE` 与 `get_current_prompt_template()`（当前 HR 模板在 `config.py`）。

7. 前端（`frontend/`）

- **Vite + Vue 3**： 对话 UI、流式解析 SSE、`conversation_id` 多轮回传、可选 `VITE_API_KEY` 请求头。
 - 开发态通过 **Vite 代理** 将 `/api` 转到后端 `8000`。
 - 「新对话」清空前端状态并重置 `conversation_id`。
- （若存在独立「文档管理」页，逻辑为：列表 `manifest`、上传、勾选、调用 `rebuild`；与上述 API 一致。）

8. 评测子系统（`ragas/`）

- 与主项目共用 `config` 与 `RAGEngine`（或独立建库脚本如 `rebuild_index.py`）。
- 使用 `answer_only` 等字段做 RAGAS，避免推理前缀干扰部分指标。
- 入口可为 `run_eval.py` → `ragas/quick_eval.py` 等（以仓库当前脚本为准）。

9. 扩展与生产建议（摘要）

- **向量库**： 单机 `FAISS` 可继续用；多实例或运维要求高时可迁 **Milvus / Qdrant / pgvector**，需改写 `KnowledgeBase/RAGEngine` 的 `vectorstore` 实现。
- **文档元数据**： 可将 `manifest` 迁 MySQL，与上传接口同事务写入。
- **鉴权**： 当前为 **API Key + 默认用户**；可扩展为 JWT、多租户 `tenant_id` 挂在会话表上。
- **运维**： `GET /health` 关注 `ok` 与 `database`；备份 **MySQL + FAISS 目录 + uploaded_docs**；`.env` 与模型目录按 `.gitignore` 不提交。

10. 关键文件速查

config.py	# 全局配置与 Prompt
knowledge_base.py	# 嵌入、切块、FAISS 读写
rag_engine.py	# 检索链、LLM、后处理、流式
doc_store.py	# 上传清单与按 id 重建索引
api_server.py	# REST + SSE + 文档 API
db/	# MySQL ORM、鉴权、聊天持久化
main.py	# CLI
frontend/	# Vue 3
ragas/	# 评测脚本与数据
docs/	# 制度源文档（默认）与本架构说明

文档版本随代码迭代；若接口或表结构变更，请同步更新本节。